École Nationale des Sciences Appliquées de Tétouan
Filière : Génie Informatique

Gestion des factures d'eau et d'électricité

Version Alpha

Mini Projet 1 – AquaVolt

Réalisé par :

Abdellah Bouarguan
Nafisa El Younoussi
Reda Zakaria
Nizar Ben Yacoub
Aymen Ouggourni

Encadrant : Mohammed AL ACHHAB

Année universitaire : 2026–2027
Date de remise : 22/10/2026

Remerciements

Nous tenons à exprimer notre gratitude à notre encadrant, **Mohammed AL ACHHAB**, pour son accompagnement, ses conseils et sa disponibilité tout au long de ce mini-projet.

Nous remercions également l'ensemble des membres de l'équipe pour leur implication, leur collaboration et le travail fourni durant les phases de conception, de développement et de validation.

Enfin, nous remercions l'École Nationale des Sciences Appliquées de Tétouan pour la formation et les moyens mis à notre disposition.

Résumé

AquaVolt est une application web de gestion des consommations d'eau et d'électricité et de leurs factures. Développée en PHP 8 avec une base MariaDB, elle repose sur une architecture MVC qui sépare l'accès aux données, le traitement métier et l'interface utilisateur.

La version Alpha couvre trois fonctionnalités principales : la gestion des clients, la saisie des consommations avec téléversement des photos des compteurs, et la gestion des factures. Elle propose également l'authentification des clients et du fournisseur, la détection des anomalies de saisie, un flux de confirmation ou de correction, ainsi que la génération automatique et la consultation des factures.

Le rapport présente le contexte du projet, l'analyse des besoins, l'architecture, le modèle de données, les modules réalisés, les mécanismes de sécurité, les tests effectués et les perspectives d'évolution.

Mots-clés : PHP, MariaDB, MVC, consommation, facturation, anomalies, sécurité, application web.

Abstract

AquaVolt is a web application for managing water and electricity consumption and the related invoices. It is developed in PHP 8 with a MariaDB database and follows an MVC architecture that separates data access, business logic and the user interface.

The Alpha version covers three main features : customer management, monthly consumption entry with meter photographs, and invoice management. It also provides role-based authentication, anomaly detection, a confirmation or correction workflow, and automatic invoice generation and consultation.

This report describes the project context, requirements analysis, architecture, data model, implemented modules, security mechanisms, tests and future improvements.

Keywords : PHP, MariaDB, MVC, consumption, billing, anomalies, security, web application.

Table des matières

Remerciements i

Résumé ii

Abstract ii

Introduction générale 1

1 Présentation du projet 2

2 Analyse des besoins 4

3 Architecture et technologies 6

4 Conception de la base de données 7

5 Gestion des clients 10

6 Saisie des consommations et gestion des anomalies 12

7 Gestion des factures 15

8 Sécurité et qualité 17

9 Tests et résultats 17

10 Bilan et perspectives 19

Conclusion générale 20

Bibliographie 21

Glossaire 21

A Routes de l'application 22

B Schéma SQL 22

Introduction générale

La gestion des consommations d'eau et d'électricité repose sur la collecte régulière des index des compteurs, la vérification des valeurs saisies et le calcul des montants à facturer. Une application web permet de centraliser ces opérations, de conserver les justificatifs photographiques et de réduire les erreurs de saisie.

Le projet **AquaVolt** a pour objectif de fournir une application permettant aux clients de transmettre leurs relevés mensuels et de consulter leurs factures, tandis que le fournisseur gère les abonnés, contrôle les anomalies et consulte les factures générées.

La version Alpha se concentre sur trois modules : la gestion des clients, la saisie des consommations et la gestion des factures. Les réclamations, les paiements, l'import annuel et le tableau de bord avancé sont réservés à une version ultérieure.

Le présent rapport décrit le contexte et les besoins du projet, l'architecture MVC adoptée, le modèle de données, les fonctionnalités développées, les mesures de sécurité, les tests réalisés et les perspectives d'amélioration.

1 Présentation du projet

1.1 Contexte

Le suivi des consommations d'eau et d'électricité nécessite la relève mensuelle des index des compteurs. Une saisie incorrecte peut entraîner des écarts importants et conduire à une facturation erronée. Le fournisseur doit donc disposer d'un outil fiable pour enregistrer les relevés, conserver les photos justificatives, détecter les valeurs incohérentes et préparer les factures.

L'application AquaVolt propose une plateforme centralisée permettant au client de saisir ses index et au fournisseur de gérer les abonnés, de contrôler les anomalies et de consulter les factures.

1.2 Problématique

La problématique principale consiste à garantir la fiabilité des consommations déclarées et à organiser leur validation avant la facturation. L'application doit notamment répondre aux questions suivantes :

— comment enregistrer les index mensuels des compteurs ?
— comment vérifier la cohérence des valeurs saisies ?
— comment conserver les photos des compteurs ?
— comment permettre la correction des relevés incorrects ?
— comment générer et consulter les factures de manière fiable ?

1.3 Objectifs

Les objectifs de la version Alpha sont les suivants :

— authentifier les clients et le fournisseur ;
— gérer les clients : ajout, modification, recherche et consultation ;
— permettre la saisie mensuelle des index d'eau et d'électricité ;
— téléverser et conserver les photos des compteurs ;
— détecter les anomalies de saisie et proposer un flux de confirmation ou de correction ;
— calculer les montants selon les tranches tarifaires et la TVA ;
— générer automatiquement les factures et permettre leur consultation ;
— contrôler l'accès aux données selon le rôle connecté.

1.4 Périmètre de la version Alpha

Table 1 – Fonctionnalités incluses et exclues de la version Alpha

Inclus dans l'Alpha	**Exclu de l'Alpha**
Authentification client et fournisseur	Inscription autonome du client
Gestion des clients	Gestion multi-fournisseurs
Saisie des consommations	Import annuel des consommations
Téléversement des photos	Réclamations
Détection et confirmation des anomalies	Paiements
Génération automatique des factures	Export PDF des factures
Consultation des factures	Notifications par e-mail
Contrôle d'accès par rôle	Tableau de bord avancé

1.5 Acteurs du système

Deux acteurs interviennent dans la version Alpha :

— **Client** : il se connecte, consulte son tableau de bord, saisit ses index mensuels, téléverse les photos de ses compteurs, confirme ou corrige une anomalie et consulte ses factures.

— **Fournisseur** : il se connecte, gère les clients, consulte les anomalies et les factures de ses abonnés.

L'acteur *Agent*, chargé de l'import annuel, est prévu dans la conception mais n'est pas implémenté dans l'Alpha.

1.6 Déroulement du projet et réunions

Le projet a été mené à travers plusieurs réunions de conception et d'implémentation. Les dates et durées sont récapitulées dans le tableau 2. Toutes les réunions ont réuni l'ensemble de l'équipe.

Table 2 – Réunions de travail du projet

Date	**Durée**	**Canal**	**Objet**
10/10/2026	2 h	Google Meet	Analyse de l'énoncé et attribution du diagramme de cas d'utilisation.
11/10/2026	2 h	Google Meet	Revue du diagramme de cas d'utilisation et des approches de diagrammes de séquence.
12/10/2026	4 h	Présentiel	Critique croisée des diagrammes, maquettes et schéma entité-relation.
14/10/2026	3 h	Présentiel	Validation de la conception et démarrage de la rédaction du rapport.
16/10/2026	2 h	Présentiel	Réunion de validation de la conception et du rapport.
18/10/2026	2 h	Google Meet	Lancement de l'implémentation de la version Alpha.
19/10/2026	30 min	Google Meet	Revue de code et intégration des branches.
20/10/2026	1 h	Google Meet	Intégration des modules clients et consommation.
21/10/2026	2 h	Google Meet	Finalisation des factures et tests de bout en bout.

1.7 Planning d'implémentation

Table 3 – Planning d'implémentation sur quatre jours

Jour	**Objectif principal**	**Livrable**
J1	Dépôt Git, structure du projet, schéma SQL, routeur et layout.	Application qui démarre et base créée.
J2	Authentification, gestion des clients, validation et CSRF.	Connexion et CRUD clients fonctionnels.
J3	Saisie des consommations, photos, anomalies et facturation.	Flux saisie → anomalie → facture.
J4	Intégration, tests, corrections UI, documentation et release.	Version Alpha stable et documentée.

1.8 Répartition des tâches

Table 4 – Répartition des tâches entre les membres

Membre	Rôle principal	Contribution
Abdellah Bouarguan	Coordination, architecture, Git	Modèle de données, SQL, structure, intégration, facturation, corrections UI et documentation.
Nafisa El Younoussi	Conception et interface	Travail conjoint sur l'authentification, la consommation, les anomalies, l'interface et le rapport.
Reda Zakaria	Gestion des clients	Modèle `ClientBD`, contrôleur client, vues, validation et recherche.
Nizar Ben Yacoub	Conception et développement	Travail conjoint sur l'authentification, la consommation, les anomalies, l'interface et le rapport.
Aymen Ouggourni	Conception et documentation	Diagrammes de séquence, participation aux vues et tests de facturation, vérifications et rapport.

2 Analyse des besoins

2.1 Besoins fonctionnels

Les besoins fonctionnels de la version Alpha sont regroupés dans le tableau 5.

Table 5 – Besoins fonctionnels de la version Alpha

ID	Besoin	Description
BF1	Authentification	Connexion du client et du fournisseur avec gestion de session et de rôle.
BF2	Gestion des clients	Ajouter, modifier, rechercher et consulter les clients.
BF3	Saisie des consommations	Saisir les index d'eau et d'électricité pour la période courante.
BF4	Photos des compteurs	Téléverser une photo pour chaque compteur.
BF5	Affichage du mois précédent	Afficher les index et les photos du relevé précédent.
BF6	Détection des anomalies	Détecter un index inférieur ou un dépassement du seuil fournisseur.
BF7	Confirmation ou correction	Permettre au client de confirmer une anomalie de seuil ou de corriger un index inférieur.
BF8	Génération des factures	Calculer les montants et enregistrer la facture après validation.
BF9	Consultation des factures	Afficher la liste et le détail des factures selon le rôle.
BF10	Contrôle d'accès	Limiter chaque utilisateur à ses propres données ou à celles de ses clients.

2.2 Besoins non fonctionnels

— **Sécurité** : requêtes préparées, protection CSRF, échappement HTML, contrôle des rôles et des uploads.
— **Fiabilité** : validation serveur des données et transactions pour les opérations multiples.
— **Maintenabilité** : séparation MVC, conventions de nommage et code organisé par module.
— **Utilisabilité** : interface claire, messages d'erreur explicites et navigation adaptée au rôle.
— **Responsive** : utilisation sur desktop, tablette et mobile.
— **Traçabilité** : conservation des consommations, photos, anomalies et factures.

2.3 Règles métier

— la saisie est ouverte du 18 du mois jusqu'au dernier jour du mois ;
— les index doivent être numériques et positifs ou nuls ;
— l'index courant doit être supérieur ou égal à l'index précédent ;
— un index supérieur au seuil du fournisseur déclenche une anomalie de seuil ;
— un index inférieur au relevé précédent impose une correction ;

— une anomalie de seuil peut être confirmée ou corrigée par le client ;
— une consommation validée ou une anomalie confirmée génère automatiquement une facture ;
— la TVA est fixée à 18 % ;
— les tarifs sont appliqués par tranches progressives ;
— une consommation ne peut produire qu'une seule facture.

2.4 Cas d'utilisation

La figure 1 présente le diagramme de cas d'utilisation du projet. La version Alpha couvre principalement l'authentification, la saisie des consommations, le traitement des anomalies et la gestion des clients. Les cas d'utilisation liés aux réclamations, à l'import annuel et aux paiements sont conservés dans la conception mais ne sont pas implémentés dans cette version.

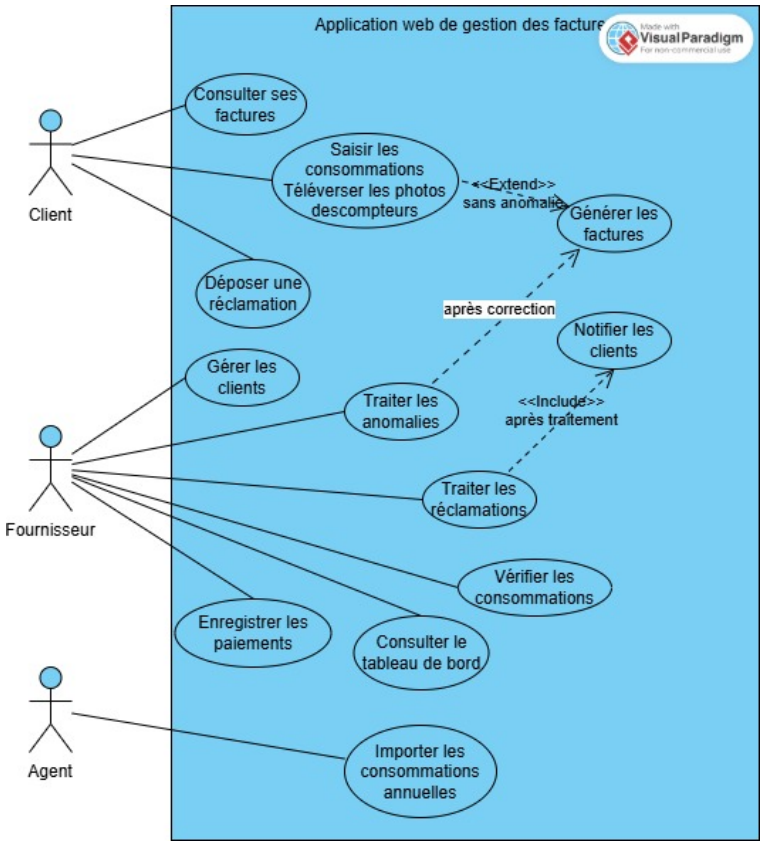


FIGURE 1 – Diagramme de cas d'utilisation du projet

TABLE 6 – Cas d'utilisation couverts par l'Alpha

Réf.	Cas d'utilisation	Acteur principal
UC01	Saisir les consommations, téléverser les photos et générer la facture	Client
UC02	Confirmer ou corriger une anomalie de saisie	Client / Fournisseur
UC04	Gérer les clients (ajouter, modifier, rechercher)	Fournisseur
—	Consulter les factures et leur détail	Client / Fournisseur

3 Architecture et technologies

3.1 Architecture MVC

Le projet suit l'architecture **MVC** (Modèle–Vue–Contrôleur), qui sépare l'accès aux données, la logique métier et la présentation. Cette organisation facilite la maintenance, les tests et l'ajout de nouvelles fonctionnalités.

— **Modèle** : couche d'accès aux données. Elle contient les classes et fonctions qui exécutent les requêtes SQL. Les fichiers principaux sont :
  — `BD/Connexion.php` ;
  — `BD/ClientBD.php` ;
  — `BD/ConsommationBD.php` ;
  — `BD/AnomalieBD.php` ;
  — `BD/Facture.php`.

— **Contrôleur** : couche de traitement. Il reçoit la requête, vérifie les droits, valide les données, applique les règles métier, appelle le modèle puis transmet les résultats à la vue. Les contrôleurs sont dans `Traitement/`.

— **Vue** : couche de présentation. Elle contient le HTML et le PHP d'affichage. Les vues sont dans `IHM/` et ne réalisent aucune requête SQL.

3.2 Routeur et point d'entrée

Toutes les requêtes passent par `index.php`. Le fichier `.htaccess` redirige vers ce point d'entrée les URLs qui ne correspondent pas à un fichier réel. Le routeur découpe l'adresse sous la forme `/Contrôleur/action/id`.

Par exemple, `/Client/update/5` charge `Traitement/Client.php` et appelle la fonction `client_update(5)`. Avant l'appel, le routeur vérifie que le contrôleur appartient à une liste autorisée, que le nom de l'action est valide et que la fonction existe. Dans le cas contraire, il affiche une page 404.

```php
$url = trim((string) ($_GET['url'] ?? ''), '/');
$segments = $url === '' ? [] : explode('/', $url);

$controller = ucfirst(strtolower($segments[0] ?? 'Home'));
$action = strtolower($segments[1] ?? 'index');
$id = $segments[2] ?? null;

$allowed = ['Home', 'Auth', 'Client', 'Consommation', 'Facture', 'Anomalie'];

if (!in_array($controller, $allowed, true)) {
    http_response_code(404);
    require ROOT . 'IHM/404.php';
    exit;
}

require_once ROOT . 'Traitement/' . $controller . '.php';
$function = strtolower($controller) . '_' . $action;
$function($id);
```

Listing 1 – Extrait simplifié du routeur

3.3 Organisation des fichiers

Le dossier `BD/` contient le modèle, `Traitement/` les contrôleurs et `IHM/` les vues. Les ressources statiques sont dans `assets/`, les photos dans `uploads/compteurs/`, le schéma SQL dans `sql/` et la documentation dans `docs/`.

3.4 Technologies utilisées

Table 7 – Technologies du projet

Technologie	Utilisation
PHP 8	Logique métier, contrôleurs, sessions et traitement des requêtes.
MariaDB / MySQL	Stockage des clients, consommations, anomalies et factures.
PDO et mysqli	Accès aux données avec requêtes préparées.
HTML / CSS	Structure et mise en forme de l'interface responsive.
JavaScript	Interactions légères et aperçu des images.
Apache ou serveur PHP	Exécution locale de l'application.
Git / GitHub	Versionnement, branches, Pull Requests et collaboration.
PlantUML / Figma	Diagrammes de conception et maquettes d'interface.

3.5 Authentification et sessions

L'authentification repose sur les tables `client` et `fournisseur`. Les mots de passe sont hachés avec `password_hash()` et vérifiés avec `password_verify()`. Après connexion, la session contient le rôle et l'identifiant correspondant :

Après connexion, la session contient le rôle, l'identifiant du client ou du fournisseur, le nom, le prénom et l'email. Les contrôleurs vérifient le rôle avant chaque action sensible. Un client ne peut accéder qu'à ses propres consommations et factures ; le fournisseur accède aux clients et aux factures de son périmètre.

3.6 Organisation Git et GitHub

Le projet est versionné avec Git et hébergé sur GitHub. Les branches principales sont :

Table 8 – Branches, Pull Requests et versions

Élément	Rôle
`main`	Branche stable contenant la version Alpha finale.
`nizar`	Branche de développement des modules clients, consommation et anomalies.
`reda`	Branche de travail sur la gestion des clients ; non fusionnée en l'état.
`aymane`	Branche de finalisation de l'Alpha et de la facturation.
`fix/alpha-ui`	Branche des corrections UI/UX et responsive.
PR #1	Fusion de la version Alpha dans `main`.
PR #2	Fusion des corrections UI/UX dans `main`.
`v0.1-alpha`	Première version Alpha.
`v0.1.1-alpha`	Version Alpha avec corrections UI/UX.

4 Conception de la base de données

4.1 Présentation

La base de données utilisée est `miniproject_version_alpha`. Elle contient sept tables principales : `fournisseur`, `client`, `tarif`, `consommation`, `photo_capteur`, `anomalie` et `facture`. Le modèle complet est présenté sur la figure 2.

4.2 Description des tables

Table 9 – Rôle des tables de la base de données

Table	Description
`fournisseur`	Informations du fournisseur, identifiants de connexion et seuils de vigilance pour l'eau et l'électricité.

Table	Description
client	Informations personnelles, CIN, adresse, email, mot de passe et fournisseur associé.
tarif	Tranches tarifaires de l'eau et de l'électricité, avec bornes et prix unitaire.
consommation	Index mensuels, période, date de saisie et état du relevé.
photo_capteur	Chemins des photos EAU et ELEC associées à une consommation.
anomalie	Type d'anomalie, champ concerné, valeurs de saisie et de référence, état et correction.
facture	Numéro, période, consommations, montants HT/TVA/TTC et état de la facture.

4.3 Relations et cardinalités

Table 10 – Relations principales du modèle

Relation	Cardinalité	Implémentation
Fournisseur – Client	1..1 – 0..n	client.id_fournisseur
Fournisseur – Tarif	1..1 – 0..n	tarif.id_fournisseur
Fournisseur – Anomalie	1..1 – 0..n	anomalie.id_fournisseur
Client – Consommation	1..1 – 0..n	consommation.id_client
Client – Facture	1..1 – 0..n	facture.id_client
Consommation – Anomalie	1..1 – 0..n	anomalie.id_conso
Consommation – Facture	1..1 – 0..1	facture.id_conso UNIQUE
Consommation – Photo	1..1 – 2..2	photo_capteur.id_conso UNIQUE(id_conso,type)

La version Alpha utilise un fournisseur unique. La colonne `id_fournisseur` est néanmoins conservée dans les tables `client`, `tarif` et `anomalie` pour préserver l'intégrité référentielle et permettre une évolution vers plusieurs fournisseurs.

4.4 Intégrité et contraintes

— moteur **InnoDB** et encodage `utf8mb4` ;
— clés étrangères avec `ON DELETE RESTRICT` pour les données maîtres ;
— `ON DELETE CASCADE` pour les photos et anomalies liées à une consommation ;
— unicité du CIN et de l'email du client ;
— unicité de l'email du fournisseur ;
— unicité de la période de consommation par client : `(id_client, periode_mois, periode_annee)` ;
— unicité du type de photo par consommation : `(id_conso, type)` ;
— unicité du numéro de facture et de la consommation facturée ;
— contraintes de vérification sur les index, les montants et les bornes tarifaires.

4.5 États et tarifs

Table 11 – États utilisés dans la base

Table	États
consommation	EN_ATTENTE, ANOMALIE, VALIDEE, FACTUREE
anomalie	EN_ATTENTE, CONFIRMEE, CORRIGEE
facture	IMPAYEE, PAYEE

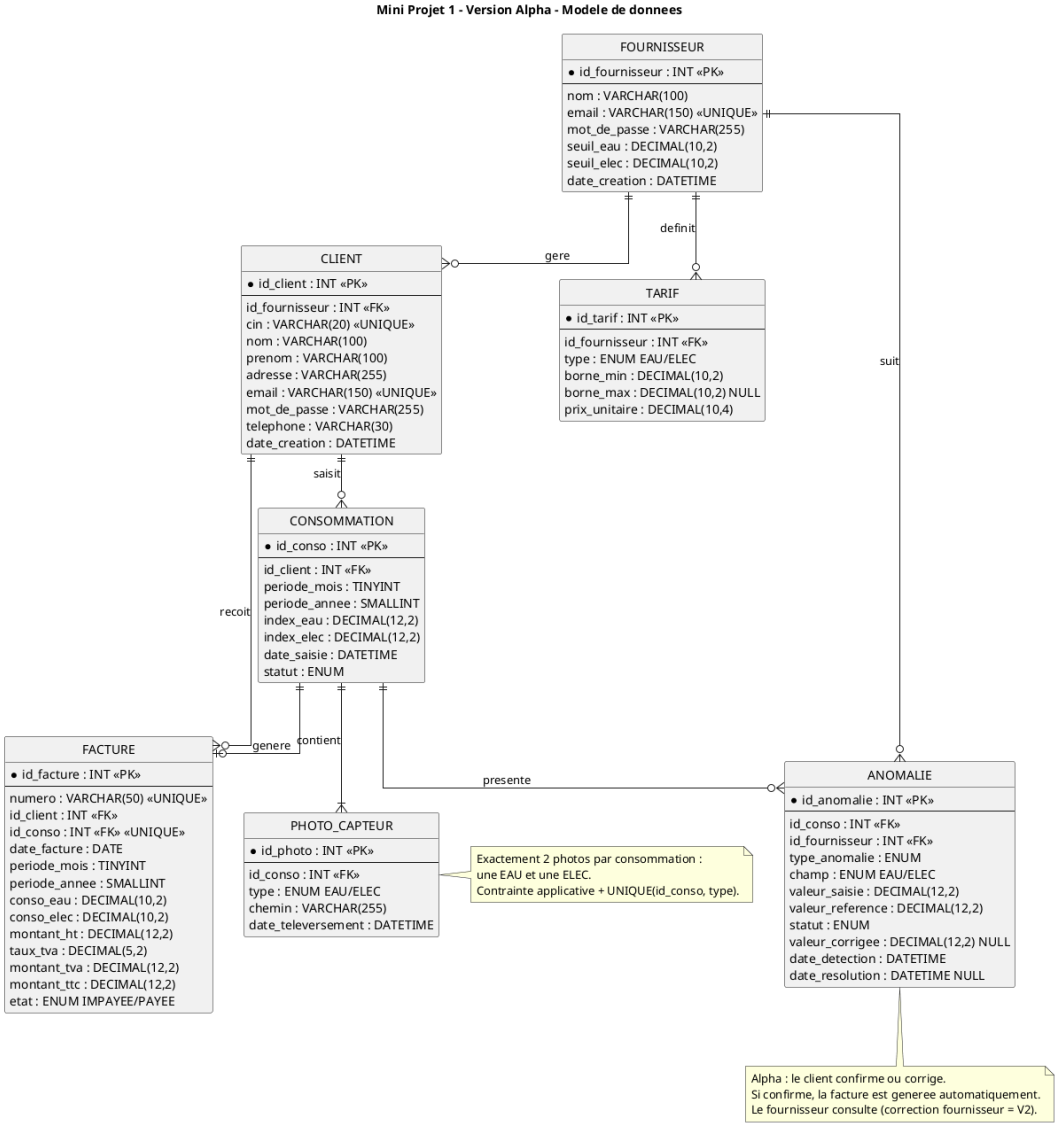


FIGURE 2 – Modèle de données de la version Alpha

Table 12 – Tranches tarifaires appliquées

Type	Tranche 1	Tranche 2	Tranche 3+
Eau	0–6 m^3 : 2,37 DH	6–20 m^3 : 7,39 DH	20–35 : 10,98 DH ; >35 : 11,03 DH
Électricité	0–100 kWh : 0,82 DH	100–150 kWh : 0,92 DH	>150 kWh : 1,10 DH

La TVA appliquée est de 18 %. Les montants sont arrondis à deux décimales.

4.6 Jeu de données de démonstration

Le script `sql/seed.sql` insère un fournisseur, trois clients, sept tarifs et une consommation de septembre 2026 pour chaque client. Ce jeu de données permet de tester la saisie du mois suivant, l'affichage du relevé précédent et la génération des factures.

5 Gestion des clients

5.1 Objectif du module

La gestion des clients permet au fournisseur de consulter la liste des abonnés, d'afficher la fiche d'un client, d'ajouter un nouveau client, de modifier ses informations et de le supprimer lorsque les règles d'intégrité le permettent.

Le module suit l'architecture MVC :

— **Modèle** : `BD/ClientBD.php` ;
— **Contrôleur** : `Traitement/Client.php` ;
— **Vues** : `IHM/client/index.php`, `one.php`, `add.php`, `update.php` et `form.php`.

5.2 Modèle : la classe ClientBD

La classe `ClientBD` est la seule couche du module à contenir du SQL. Chaque méthode correspond à une opération sur la table `client` :

— `lister()` : liste les clients d'un fournisseur, avec recherche ;
— `trouver()` : retourne un client selon son identifiant et son fournisseur ;
— `creer()` : insère un nouveau client ;
— `mettreAJour()` : met à jour les informations d'un client ;
— `supprimer()` : supprime un client ;
— `emailExiste()` et `cinExiste()` : vérifient l'unicité ;
— `nbConsommations()` : compte les consommations d'un client.

Toutes les requêtes sont préparées. Les valeurs ne sont jamais concaténées directement dans le texte SQL, ce qui protège contre les injections SQL.

5.3 Contrôleur : Traitement/Client.php

Le contrôleur est le chef d'orchestre du module. Chaque action est une fonction :

— `client_all()` : affiche la liste et gère la recherche ;
— `client_one()` : affiche la fiche d'un client ;
— `client_add()` : affiche puis traite le formulaire d'ajout ;
— `client_update()` : affiche puis traite le formulaire de modification ;
— `client_delete()` : supprime un client si les règles d'intégrité le permettent.

Des fonctions d'appui complètent ces actions : `client_exiger_fournisseur()` vérifie que l'utilisateur est un fournisseur connecté, `client_lire_formulaire()` nettoie et valide les champs, et `client_csrf_valide()` contrôle le jeton CSRF.

5.4 Vues : IHM/client

Les vues ne contiennent que du HTML et du PHP d'affichage. Elles ne font aucune requête SQL. Elles reçoivent leurs données du contrôleur sous forme de variables : liste des clients, fiche client, messages d'erreur ou valeurs saisies.

Le formulaire `form.php` est partagé entre l'ajout et la modification, ce qui évite de dupliquer le code. Les pages `add.php` et `update.php` fournissent le layout, le titre et la carte autour du formulaire. Toute valeur affichée est échappée avec la fonction `e()` pour empêcher les failles XSS.

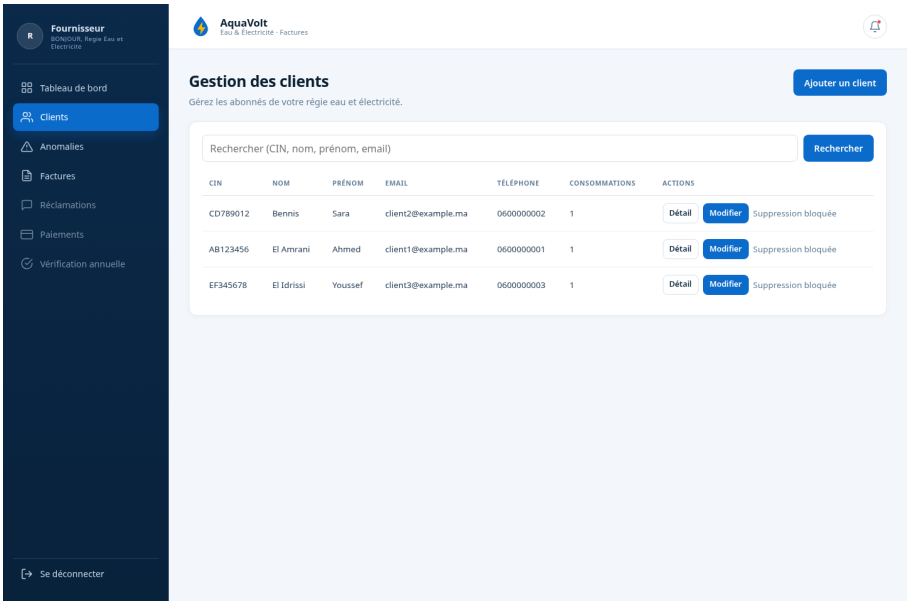


FIGURE 3 – Liste des clients avec recherche

5.5 Déroulement d'une requête d'ajout

Lorsque le fournisseur ouvre `/Client/add`, le contrôleur affiche le formulaire vide. Lors de l'envoi :

1. le contrôleur vérifie le rôle de l'utilisateur ;
2. il vérifie le jeton CSRF ;
3. il valide chaque champ : CIN, nom, prénom, adresse, email, téléphone et mot de passe ;
4. il vérifie que l'email et le CIN n'existent pas déjà ;
5. il hache le mot de passe avec `password_hash()` ;
6. il demande au modèle d'enregistrer le client ;
7. il redirige vers la liste avec un message de succès.

En cas d'erreur, il réaffiche le formulaire avec les messages et les valeurs déjà saisies. Le contrôleur ne fait jamais de SQL, le modèle n'affiche jamais de HTML et la vue ne prend aucune décision métier.

5.6 Validation et unicité

Les champs obligatoires sont le CIN, le nom, le prénom, l'adresse et l'email. Le CIN et l'email doivent être uniques. Le mot de passe doit contenir au moins huit caractères lors de la création. Les doublons sont détectés avant l'insertion et une contrainte d'unicité dans la base assure également l'intégrité.

La recherche porte sur le CIN, le nom, le prénom et l'email. Elle utilise une requête préparée avec des paramètres `LIKE`.

5.7 CSRF et suppression conditionnelle

Les formulaires d'ajout, de modification et de suppression contiennent un jeton CSRF. Le contrôleur vérifie ce jeton avant toute écriture.

La suppression est bloquée lorsqu'un client possède au moins une consommation. Le contrôleur vérifie le nombre de consommations avant de supprimer et la clé étrangère `ON DELETE RESTRICT` empêche toute suppression incohérente au niveau de la base. L'interface affiche « Suppression bloquée » à la place du bouton pour ces clients.

5.8 Tests du module

Les tests réalisés sur le module clients couvrent :

— l'accès à la liste et la recherche ;
— l'ajout d'un client valide ;
— le refus d'un CIN ou d'un email en doublon ;
— la modification des informations et du mot de passe ;
— le blocage de la suppression d'un client ayant des consommations ;
— le contrôle du rôle fournisseur et du jeton CSRF.

6 Saisie des consommations et gestion des anomalies

6.1 Fonctionnalités

Le module de consommation permet au client de saisir les index de son compteur d'eau et de son compteur électrique, de téléverser les photos des compteurs et de consulter l'historique de ses relevés. Il assure également la détection des anomalies et le flux de confirmation ou de correction avant la génération de la facture.

La figure 4 présente le scénario principal de saisie, de détection d'anomalie et de génération de la facture.

6.2 Période de saisie

La saisie est autorisée du 18 du mois jusqu'au dernier jour du mois. Le contrôleur vérifie que la période correspond au mois en cours et que le jour courant est supérieur ou égal à 18. Une saisie hors période est refusée avec un message explicite.

6.3 Formulaire et photos

Le formulaire présente deux cartes : une pour le compteur d'eau et une pour le compteur électrique. Pour chaque compteur, le client saisit le nouvel index et téléverse une photo. Les photos du mois précédent sont affichées lorsqu'elles existent.

Les fichiers sont contrôlés avant enregistrement :

— présence des deux photos ;
— taille maximale de 5 Mo par photo ;
— formats acceptés : JPG, PNG et WEBP ;
— vérification du type MIME réel avec `finfo` ;
— génération d'un nom de fichier aléatoire ;
— stockage dans `uploads/compteurs/`.

Lors d'une correction, les photos existantes sont conservées si le client n'en téléverse pas de nouvelles.

6.4 Calcul de la consommation

La consommation est calculée par différence entre l'index courant et le dernier index validé ou facturé :

$$C = I_{\text{actuel}} - I_{\text{précédent}}$$

Si aucun relevé précédent n'existe, l'index de référence est zéro. La consommation d'eau est exprimée en m^3 et celle d'électricité en kWh.



FIGURE 4 – Diagramme de séquence de la saisie d'une consommation

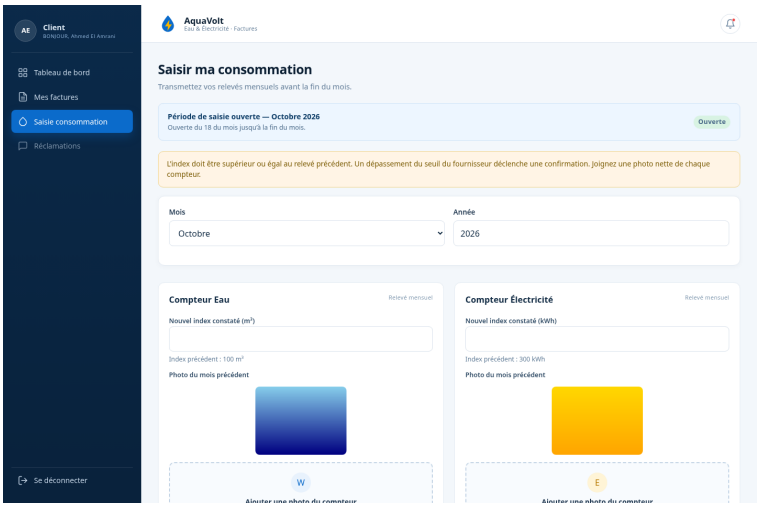


FIGURE 5 – Formulaire de saisie avec relevé précédent et photos

6.5 Détection des anomalies

Deux types d'anomalies sont détectés :

— `INDEX_INFERIEUR` : l'index courant est inférieur à l'index précédent ;
— `SEUIL_DEPASSE` : l'index courant dépasse le seuil défini par le fournisseur.

Le seuil est celui enregistré dans la table `fournisseur` : `seuil_eau` pour l'eau et `seuil_elec` pour l'électricité. La règle des +40 % présente dans certaines maquettes n'est pas retenue dans la version Alpha.

6.6 Confirmation et correction

Le traitement dépend du type d'anomalie :

— un `INDEX_INFERIEUR` doit obligatoirement être corrigé ;
— un `SEUIL_DEPASSE` peut être confirmé ou corrigé par le client ;
— après confirmation, l'anomalie passe à l'état `CONFIRMEE` et la consommation à `VALIDEE` ;
— après correction, l'ancienne anomalie passe à l'état `CORRIGEE`, puis le relevé est contrôlé à nouveau ;
— après validation, la facture est générée automatiquement.

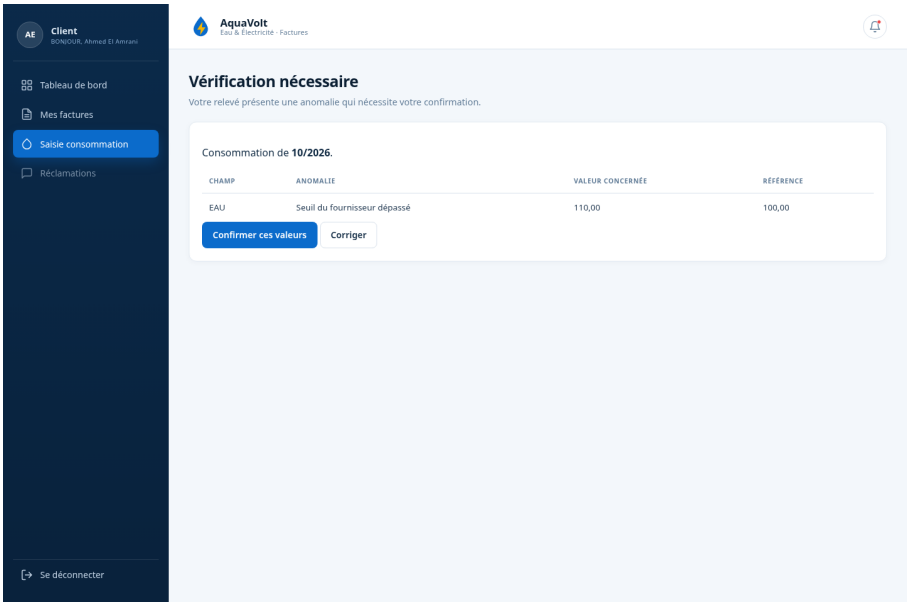


FIGURE 6 – Confirmation d'une anomalie de seuil

Le fournisseur consulte les anomalies en attente, mais la correction est réalisée par le client dans le flux de la version Alpha.

6.7 Historique et détail

Le client peut consulter l'historique de ses consommations : période, index saisis, consommations calculées, statut, facture associée et actions disponibles. La page de détail présente les photos, les anomalies et la facture générée.

6.8 Tests du module

Les tests couvrent la période de saisie, les valeurs négatives ou non numériques, les photos manquantes ou invalides, la détection d'un index inférieur, la détection d'un dépassement de seuil, la confirmation, la correction et la génération de la facture.

7 Gestion des factures

7.1 Objectif du module

Le module de facturation transforme une consommation validée en facture enregistrée. Il calcule les montants selon les tranches tarifaires, applique la TVA de 18 %, génère un numéro unique, enregistre la facture et permet sa consultation par le client et le fournisseur.

7.2 Tarification

Les tarifs sont stockés dans la table `tarif`. Chaque ligne contient un type (`EAU` ou `ELEC`), une borne minimale, une borne maximale et un prix unitaire. Les tranches sont appliquées de manière progressive.

TABLE 13 – Tranches tarifaires de l'eau et de l'électricité

Type	Tranche 1	Tranche 2	Tranche 3
Eau	0–6 m^3 : 2,37 DH	6–20 : 7,39 DH	20–35 : 10,98 DH ; >35 : 11,03 DH
Électricité	0–100 kWh : 0,82 DH	100–150 : 0,92 DH	>150 : 1,10 DH

Pour chaque tranche, le montant est calculé sur la quantité effectivement consommée dans l'intervalle :

$$M = \sum_{t} (\min(C, borne_{max,t}) - borne_{min,t}) \times prix_t$$

7.3 TVA et arrondis

La TVA est fixée à 18 %. Les montants sont arrondis à deux décimales :

$$HT = M_{eau} + M_{elec}, \qquad TVA = HT \times 0{,}18, \qquad TTC = HT + TVA$$

7.4 Génération de la facture

La génération est prise en charge par `Traitement/FactureService.php`. Les étapes sont les suivantes :

1. vérifier que la consommation est à l'état `VALIDEE` ;
2. vérifier qu'aucune facture n'existe déjà pour cette consommation ;
3. récupérer le client, le fournisseur, les index et les tarifs ;
4. calculer les consommations d'eau et d'électricité ;
5. calculer les montants HT, TVA et TTC ;
6. générer le numéro `FAC-YYYYMM-{id_conso}` ;
7. insérer la facture et passer la consommation à l'état `FACTUREE` dans une transaction ;
8. annuler la transaction en cas d'échec.

```php
$this->pdo->beginTransaction();
try {
    $this->factures->create([...]);
    $this->consommations->changerStatut($idConso, 'FACTUREE');
    $this->pdo->commit();
} catch (Throwable $e) {
    if ($this->pdo->inTransaction()) {
        $this->pdo->rollBack();
    }
    throw $e;
}
```

Listing 2 – Principe de génération transactionnelle

7.5 Numéro et état de la facture

Le numéro est unique et construit à partir de la période et de l'identifiant de la consommation. L'état initial est `IMPAYEE`. Le passage à `PAYEE` et l'enregistrement d'un paiement sont réservés à une version ultérieure.

7.6 Consultation et contrôle d'accès

Le client consulte uniquement ses propres factures. Le fournisseur consulte les factures de ses clients. La liste présente le numéro, la date, la période, les consommations, le montant TTC, l'état et un lien vers le détail.

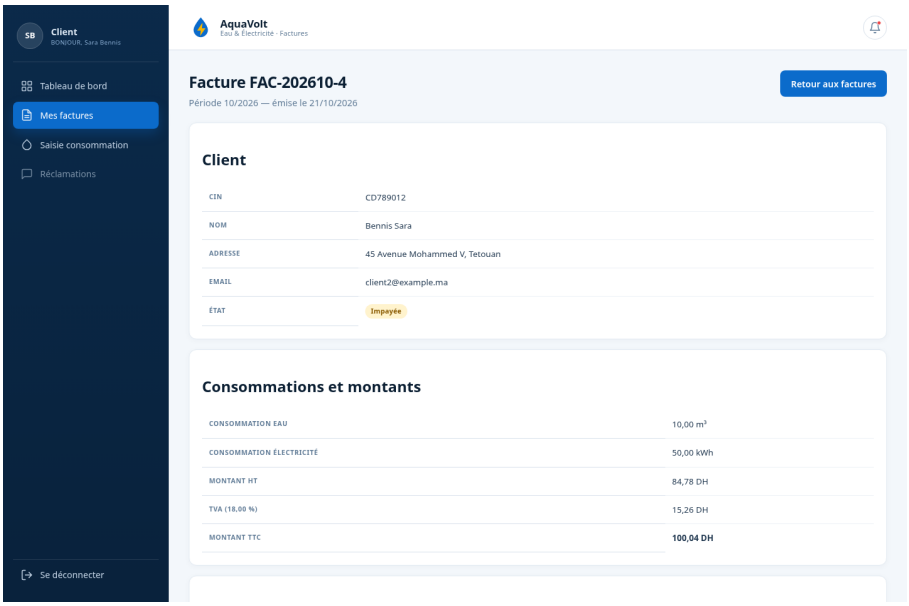


FIGURE 7 – Détail d'une facture avec montants et photos

7.7 Exemples de calcul

TABLE 14 – Exemples de calculs de facture

Eau	Élec.	HT	TVA	TTC	Cas
25 m^3	200 kWh	355,58	64,00	419,58	Référence
10 m^3	50 kWh	84,78	15,26	100,04	Saisie valide
0 m^3	50 kWh	41,00	7,38	48,38	Correction

7.8 Manuel d'utilisation

1. démarrer le serveur PHP ou Apache et ouvrir l'application ;
2. se connecter avec un compte client ou fournisseur ;
3. le fournisseur gère les clients depuis le menu *Clients* ;
4. le client saisit ses index depuis *Saisie consommation* et téléverse les deux photos ;
5. en cas d'anomalie, le client confirme ou corrige ;
6. la facture est générée automatiquement et accessible depuis *Mes factures* ;
7. le fournisseur consulte toutes les factures depuis le menu *Factures*.

7.9 Tests du module

Les tests vérifient la génération après une saisie valide, après confirmation d'une anomalie et après correction, l'unicité de la facture, les montants HT/TVA/TTC, l'accès du client à ses seules factures et l'accès du fournisseur aux factures de ses clients.

8 Sécurité et qualité

Plusieurs mécanismes ont été intégrés pour renforcer la sécurité et la cohérence de l'application :

— utilisation de requêtes préparées avec PDO ou mysqli ;
— hachage des mots de passe avec `password_hash()` et vérification avec `password_verify()` ;
— régénération de l'identifiant de session après connexion ;
— contrôle du rôle avant l'accès aux pages protégées ;
— vérification de l'appartenance des consommations et factures au client connecté ;
— protection CSRF des formulaires sensibles ;
— échappement du contenu HTML avec `htmlspecialchars()` ;
— validation des champs côté serveur ;
— contrôle du type MIME réel des photos avec `finfo` ;
— génération de noms de fichiers aléatoires ;
— limitation de l'exécution PHP dans le dossier des uploads ;
— utilisation de transactions SQL pour les opérations multiples ;
— contraintes d'unicité et clés étrangères dans la base ;
— suppression des scripts de connexion de démonstration.

Ces mécanismes limitent les erreurs de saisie, les accès non autorisés et les incohérences de données. La version Alpha ne met pas encore en place de limitation du nombre de tentatives de connexion, de vérification d'adresse email ni de réinitialisation de mot de passe.

9 Tests et résultats

9.1 Méthodologie

Les tests ont été réalisés manuellement et à l'aide d'un navigateur automatisé sur la version Alpha. Ils couvrent les parcours fonctionnels, les calculs de facturation, le contrôle d'accès, la sécurité des formulaires et le rendu responsive. La syntaxe de tous les fichiers PHP a également été vérifiée avec `php -l`.

9.2 Tests fonctionnels

Table 15 – Synthèse des tests fonctionnels

Test	Résultat	Observation
Connexion client et fournisseur	Réussi	Redirection selon le rôle.
Refus d'un mot de passe incorrect	Réussi	Message d'erreur affiché.
Accès sans session	Réussi	Redirection vers la connexion.
Accès client à une page fournisseur	Réussi	Accès refusé.
Facture d'un autre client	Réussi	Erreur 404.
Ajout d'un client valide	Réussi	Message de succès.
Refus d'un CIN ou email en doublon	Réussi	Erreurs sous les champs.
Modification d'un client	Réussi	Mot de passe conservé si vide.
Recherche d'un client	Réussi	Recherche CIN, nom, prénom, email.
Suppression d'un client avec consommation	Réussi	Suppression bloquée.
Saisie avant le 18 ou hors mois courant	Réussi	Refus serveur.
Valeurs négatives ou non numériques	Réussi	Refus serveur.
Photos manquantes ou invalides	Réussi	Refus serveur.

Test	Résultat	Observation
Saisie valide avec photos	Réussi	Consommation enregistrée.
Détection d'un index inférieur	Réussi	Anomalie `INDEX_INFERIEUR`.
Détection d'un dépassement de seuil	Réussi	Anomalie `SEUIL_DEPASSE`.
Confirmation d'une anomalie de seuil	Réussi	Facture générée.
Correction d'un index inférieur	Réussi	Revalidation puis facture.
Génération d'une facture	Réussi	Numéro unique et montants enregistrés.
Consultation client/fournisseur	Réussi	Contrôle d'accès respecté.
CSRF invalide	Réussi	Erreur 419 stylée.
Vérification syntaxique PHP	Réussi	Aucune erreur.

9.3 Calculs de référence

Les montants suivants ont été vérifiés :

— 25 m^3 d'eau et 200 kWh : HT = 355,58 DH, TVA = 64,00 DH, TTC = 419,58 DH ;
— 10 m^3 d'eau et 50 kWh : HT = 84,78 DH, TVA = 15,26 DH, TTC = 100,04 DH ;
— 0 m^3 d'eau et 50 kWh : HT = 41,00 DH, TVA = 7,38 DH, TTC = 48,38 DH.

9.4 Tests de sécurité

Les tests de sécurité ont vérifié :

— l'impossibilité d'accéder aux données d'un autre client ;
— le refus d'un jeton CSRF absent ou invalide ;
— l'échappement des valeurs affichées ;
— le contrôle du type MIME des photos ;
— l'absence de scripts de connexion de démonstration ;
— l'utilisation de requêtes préparées pour les accès à la base.

9.5 Audit UI et responsive

Un audit visuel a été réalisé sur desktop, tablette et mobile, puis comparé à la maquette Figma. Les problèmes détectés et corrigés sont les suivants :

— rétablissement du layout des pages d'ajout et de modification d'un client ;
— reconstruction du formulaire de consommation avec cartes compteur, photos et boutons cohérents ;
— affichage des photos du mois précédent ;
— correction de la navigation mobile avec une barre fixe en bas ;
— ajout de wrappers horizontaux pour les tableaux ;
— correction des pages d'erreur CSRF et 404 ;
— traduction des statuts et ajout de badges colorés ;
— correction de la grille des photos ;
— suppression du bouton de correction des anomalies côté fournisseur, conformément au flux retenu.

Les tests responsive ont été effectués aux largeurs 1440 px, 1024 px et 390 px. Après corrections, aucune page ne présente de débordement horizontal.

Les éléments restant à améliorer sont principalement l'alignement typographique exact avec la maquette Figma, les filtres et la pagination des listes, ainsi que l'enrichissement des tableaux de bord.

9.6 Limites des tests

Les tests sont principalement manuels et fonctionnels. La version Alpha ne dispose pas encore de tests unitaires automatisés pour la tarification et la génération de facture. Ces tests pourraient être ajoutés dans une version ultérieure.

10 Bilan et perspectives

10.1 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées durant le projet :

— problèmes de routage et d'URL de base entre le serveur PHP intégré et Apache ;
— coexistence de connexions PDO et mysqli dans le projet ;
— fonctions déclarées plusieurs fois lors de la fusion des branches ;
— vues manquantes ou incomplètes au moment de l'intégration ;
— base de données initialement vide et absence de données de démonstration ;
— harmonisation des seuils de vigilance et de la règle d'anomalie ;
— génération réelle des factures à finaliser après la phase de simulation ;
— corrections de mise en page et de responsive sur les formulaires et les tableaux ;
— gestion des conflits Git entre les branches de travail.

Ces difficultés ont été traitées progressivement par la correction du routage, l'organisation des fichiers, la stabilisation des branches, la reprise des données de départ et la réalisation de tests de bout en bout.

10.2 Compétences acquises

Le projet a permis de mobiliser et de renforcer les compétences suivantes :

— développement web avec PHP 8 ;
— conception d'une application selon l'architecture MVC ;
— modélisation et manipulation d'une base MariaDB ;
— utilisation de PDO et mysqli avec requêtes préparées ;
— gestion des sessions, des rôles et du contrôle d'accès ;
— validation de formulaires et protection CSRF ;
— téléversement sécurisé de fichiers ;
— transactions SQL et contraintes d'intégrité ;
— calcul de tranches tarifaires et de TVA ;
— conception d'interfaces HTML/CSS responsive ;
— utilisation de Git, GitHub, des branches et des Pull Requests ;
— rédaction d'une documentation technique et d'un rapport LaTeX.

10.3 Limites de la version Alpha

La version Alpha est fonctionnelle sur les trois modules principaux, mais elle présente les limites suivantes :

— pas d'enregistrement de paiement ni de passage automatique à l'état `PAYEE` ;
— pas d'export PDF des factures ;
— pas de réclamations ;
— pas d'import annuel ni de gestion de l'acteur Agent ;
— pas de tableau de bord avancé avec graphiques ;
— pas de filtres ni de pagination sur les listes ;
— pas de notifications par e-mail ;
— pas d'inscription autonome ni de réinitialisation de mot de passe ;
— pas de gestion multi-fournisseurs dans l'interface ;
— pas de tests unitaires automatisés.

10.4 Perspectives d'amélioration

Les évolutions envisagées sont les suivantes :

1. ajouter l'enregistrement des paiements et le passage des factures à l'état `PAYEE` ;
2. générer un PDF téléchargeable pour chaque facture ;
3. ajouter des filtres par période et par état, ainsi que la pagination ;
4. mettre en place des notifications par e-mail ;
5. implémenter les réclamations et leur traitement ;
6. implémenter l'import annuel des consommations et l'espace Agent ;
7. enrichir le tableau de bord avec des indicateurs et des graphiques ;
8. ajouter des tests unitaires et des tests de non-régression ;
9. aligner plus finement l'interface sur la maquette Figma.

Conclusion générale

Le projet AquaVolt a permis de développer une application web de gestion des consommations d'eau et d'électricité et de leurs factures. L'architecture MVC adoptée sépare l'accès aux données, la logique métier et la présentation, ce qui facilite l'organisation et la maintenance du code.

La version Alpha dispose d'une authentification par rôle, d'une gestion des clients, d'un module de saisie des consommations avec téléversement des photos, d'un mécanisme de détection et de traitement des anomalies, ainsi que d'une génération réelle et d'une consultation des factures. Les calculs de tranches tarifaires et de TVA ont été vérifiés, et les parcours principaux ont été testés.

Les tests ont confirmé le fonctionnement des trois modules principaux et le respect du contrôle d'accès. Les corrections UI ont également amélioré la cohérence visuelle et le responsive de l'application.

La version Alpha constitue ainsi une base fonctionnelle et documentée. Les perspectives portent principalement sur les paiements, l'export PDF, les réclamations, l'import annuel, les notifications et l'enrichissement du tableau de bord.

Bibliographie

[1] PHP Group. *Documentation officielle PHP*. https://www.php.net/docs.php

[2] Oracle. *MySQL Documentation*. https://dev.mysql.com/doc/

[3] MariaDB Foundation. *MariaDB Documentation*. https://mariadb.com/kb/

[4] Mozilla. *MDN Web Docs*. https://developer.mozilla.org/

[5] Figma. *Figma Design Tool*. https://www.figma.com/

[6] PlantUML. *PlantUML Language Reference Guide*. https://plantuml.com/

[7] Git. *Git Documentation*. https://git-scm.com/doc

[8] GitHub. *GitHub Docs*. https://docs.github.com/

[9] Équipe AquaVolt. *Rapport de validation et d'organisation du projet*, 2026.

[10] École Nationale des Sciences Appliquées de Tétouan. *Énoncé du Mini Projet 1 – Version Alpha*, 2026.

Glossaire

API	Interface de programmation permettant à des composants de communiquer.
CRUD	Créer, lire, mettre à jour et supprimer des données.
CSRF	Falsification de requête intersite ; protection par jeton pour les formulaires.
FK	Clé étrangère assurant l'intégrité référentielle entre deux tables.
IHM	Interface homme-machine, c'est-à-dire la couche de présentation.
MVC	Modèle–Vue–Contrôleur, architecture séparant données, présentation et traitement.
PDO	PHP Data Objects, extension d'accès aux bases de données.
SGBD	Système de gestion de base de données.
TVA	Taxe sur la valeur ajoutée, fixée à 18 % dans le projet.
XSS	Injection de code HTML ou JavaScript dans une page.

A Routes de l'application

TABLE 16 – Routes principales et contrôleurs associés

URL	Méthode	Rôle / fonction
`/Auth/login`	GET/POST	Connexion client ou fournisseur
`/Auth/logout`	GET	Déconnexion
`/Client/all`	GET	Liste et recherche des clients (fournisseur)
`/Client/one/{id}`	GET	Fiche client (fournisseur)
`/Client/add`	GET/POST	Ajout d'un client (fournisseur)
`/Client/update/{id}`	GET/POST	Modification d'un client (fournisseur)
`/Client/delete/{id}`	POST	Suppression conditionnelle (fournisseur)
`/Consommation/add`	GET/POST	Saisie d'une consommation (client)
`/Consommation/add/{id}`	GET/POST	Correction d'une consommation (client)
`/Consommation/all`	GET	Historique des consommations (client)
`/Consommation/one/{id}`	GET	Détail d'une consommation (client)
`/Consommation/confirmation/{id}`	GET	Confirmation ou correction d'une anomalie (client)
`/Consommation/confirmer`	POST	Confirmer ou corriger une anomalie (client)
`/Consommation/facturer`	POST	Génération manuelle d'une facture (client)
`/Facture/all`	GET	Liste des factures (client/fournisseur)
`/Facture/one/{id}`	GET	Détail d'une facture (client/fournisseur)
`/Anomalie/index`	GET	Consultation des anomalies (fournisseur)
`/Anomalie/corriger/{id}`	GET/POST	Correction fournisseur (non utilisée dans le flux Alpha)

B Schéma SQL

Le schéma complet de la base de données est fourni ci-dessous. Il crée la base `miniproject_version_alpha`, les sept tables, les clés étrangères et les contraintes d'intégrité.

```sql
-- ============================================================
-- Mini Projet 1 - Version Alpha
-- Base de donnees : miniproject_version_alpha
-- SGBD cible : MariaDB / MySQL (InnoDB, utf8mb4)
-- ============================================================

CREATE DATABASE IF NOT EXISTS miniproject_version_alpha
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE miniproject_version_alpha;

SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS facture;
DROP TABLE IF EXISTS anomalie;
DROP TABLE IF EXISTS photo_capteur;
DROP TABLE IF EXISTS consommation;
DROP TABLE IF EXISTS tarif;
DROP TABLE IF EXISTS client;
DROP TABLE IF EXISTS fournisseur;

SET FOREIGN_KEY_CHECKS = 1;

-- ------------------------------------------------------------
-- FOURNISSEUR
-- ------------------------------------------------------------
CREATE TABLE fournisseur (
  id_fournisseur INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nom VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  mot_de_passe VARCHAR(255) NOT NULL,
  seuil_eau DECIMAL(10,2) NOT NULL DEFAULT 100.00,
  seuil_elec DECIMAL(10,2) NOT NULL DEFAULT 1000.00,
  date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_fournisseur),
  UNIQUE KEY uk_fournisseur_email (email),
  CONSTRAINT chk_fournisseur_seuils CHECK (seuil_eau >= 0 AND seuil_elec >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
-- ------------------------------------------------------------
-- CLIENT
-- ------------------------------------------------------------
CREATE TABLE client (
  id_client INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_fournisseur INT UNSIGNED NOT NULL,
  cin VARCHAR(20) NOT NULL,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  adresse VARCHAR(255) NOT NULL,
  email VARCHAR(150) NOT NULL,
  mot_de_passe VARCHAR(255) NOT NULL,
  telephone VARCHAR(30) NULL,
  date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_client),
  UNIQUE KEY uk_client_cin (cin),
  UNIQUE KEY uk_client_email (email),
  KEY idx_client_fournisseur (id_fournisseur),
  CONSTRAINT fk_client_fournisseur
    FOREIGN KEY (id_fournisseur) REFERENCES fournisseur (id_fournisseur)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------
-- TARIF
-- borne_max NULL = derniere tranche
-- Les bornes sont continues : [borne_min, borne_max[
-- Exemple eau : 0-6, 6-20, 20-35, 35-infini
-- ------------------------------------------------------------
CREATE TABLE tarif (
  id_tarif INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_fournisseur INT UNSIGNED NOT NULL,
  type ENUM('EAU','ELEC') NOT NULL,
  borne_min DECIMAL(10,2) NOT NULL,
  borne_max DECIMAL(10,2) NULL,
  prix_unitaire DECIMAL(10,4) NOT NULL,
  PRIMARY KEY (id_tarif),
  UNIQUE KEY uk_tarif_fournisseur_type_borne (id_fournisseur, type, borne_min),
  KEY idx_tarif_fournisseur (id_fournisseur),
  CONSTRAINT fk_tarif_fournisseur
    FOREIGN KEY (id_fournisseur) REFERENCES fournisseur (id_fournisseur)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_tarif_bornes
    CHECK (borne_min >= 0 AND (borne_max IS NULL OR borne_max > borne_min)),
  CONSTRAINT chk_tarif_prix CHECK (prix_unitaire >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------
-- CONSOMMATION
-- ------------------------------------------------------------
CREATE TABLE consommation (
  id_conso INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_client INT UNSIGNED NOT NULL,
  periode_mois TINYINT UNSIGNED NOT NULL,
  periode_annee SMALLINT UNSIGNED NOT NULL,
  index_eau DECIMAL(12,2) NOT NULL,
  index_elec DECIMAL(12,2) NOT NULL,
  date_saisie DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  statut ENUM('EN_ATTENTE','ANOMALIE','VALIDEE','FACTUREE') NOT NULL DEFAULT 'EN_ATTENTE',
  PRIMARY KEY (id_conso),
  UNIQUE KEY uk_conso_client_periode (id_client, periode_mois, periode_annee),
  KEY idx_conso_client (id_client),
  CONSTRAINT fk_conso_client
    FOREIGN KEY (id_client) REFERENCES client (id_client)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_conso_mois CHECK (periode_mois BETWEEN 1 AND 12),
  CONSTRAINT chk_conso_index CHECK (index_eau >= 0 AND index_elec >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------
-- PHOTO_CAPTEUR
-- Exactement 2 photos par consommation : EAU + ELEC.
-- Le UNIQUE garantit au plus une photo par type.
-- ------------------------------------------------------------
CREATE TABLE photo_capteur (
  id_photo INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_conso INT UNSIGNED NOT NULL,
  type ENUM('EAU','ELEC') NOT NULL,
  chemin VARCHAR(255) NOT NULL,
  date_televersement DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_photo),
  UNIQUE KEY uk_photo_conso_type (id_conso, type),
  KEY idx_photo_conso (id_conso),
  CONSTRAINT fk_photo_conso
    FOREIGN KEY (id_conso) REFERENCES consommation (id_conso)
    ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
-- ------------------------------------------------------------
-- ANOMALIE
-- type_anomalie : index inferieur au mois precedent / seuil depasse
-- champ : EAU ou ELEC
-- statut : EN_ATTENTE (choix client), CONFIRMEE, CORRIGEE
-- ------------------------------------------------------------
CREATE TABLE anomalie (
  id_anomalie INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_conso INT UNSIGNED NOT NULL,
  id_fournisseur INT UNSIGNED NOT NULL,
  type_anomalie ENUM('INDEX_INFERIEUR','SEUIL_DEPASSE') NOT NULL,
  champ ENUM('EAU','ELEC') NOT NULL,
  valeur_saisie DECIMAL(12,2) NOT NULL,
  valeur_reference DECIMAL(12,2) NOT NULL,
  statut ENUM('EN_ATTENTE','CONFIRMEE','CORRIGEE') NOT NULL DEFAULT 'EN_ATTENTE',
  valeur_corrigee DECIMAL(12,2) NULL,
  date_detection DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  date_resolution DATETIME NULL,
  PRIMARY KEY (id_anomalie),
  KEY idx_anomalie_conso (id_conso),
  KEY idx_anomalie_fournisseur (id_fournisseur),
  CONSTRAINT fk_anomalie_conso
    FOREIGN KEY (id_conso) REFERENCES consommation (id_conso)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_anomalie_fournisseur
    FOREIGN KEY (id_fournisseur) REFERENCES fournisseur (id_fournisseur)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_anomalie_valeurs
    CHECK (valeur_saisie >= 0 AND valeur_reference >= 0
          AND (valeur_corrigee IS NULL OR valeur_corrigee >= 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------
-- FACTURE
-- Une facture par consommation : id_conso UNIQUE.
-- Les montants sont stockes pour conserver l'historique.
-- ------------------------------------------------------------
CREATE TABLE facture (
  id_facture INT UNSIGNED NOT NULL AUTO_INCREMENT,
  numero VARCHAR(50) NOT NULL,
  id_client INT UNSIGNED NOT NULL,
  id_conso INT UNSIGNED NOT NULL,
  date_facture DATE NOT NULL,
  periode_mois TINYINT UNSIGNED NOT NULL,
  periode_annee SMALLINT UNSIGNED NOT NULL,
  conso_eau DECIMAL(10,2) NOT NULL,
  conso_elec DECIMAL(10,2) NOT NULL,
  montant_ht DECIMAL(12,2) NOT NULL,
  taux_tva DECIMAL(5,2) NOT NULL DEFAULT 18.00,
  montant_tva DECIMAL(12,2) NOT NULL,
  montant_ttc DECIMAL(12,2) NOT NULL,
  etat ENUM('IMPAYEE','PAYEE') NOT NULL DEFAULT 'IMPAYEE',
  PRIMARY KEY (id_facture),
  UNIQUE KEY uk_facture_numero (numero),
  UNIQUE KEY uk_facture_conso (id_conso),
  KEY idx_facture_client (id_client),
  CONSTRAINT fk_facture_client
    FOREIGN KEY (id_client) REFERENCES client (id_client)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_facture_conso
    FOREIGN KEY (id_conso) REFERENCES consommation (id_conso)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_facture_mois CHECK (periode_mois BETWEEN 1 AND 12),
  CONSTRAINT chk_facture_montants
    CHECK (conso_eau >= 0 AND conso_elec >= 0
          AND montant_ht >= 0 AND montant_tva >= 0 AND montant_ttc >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Listing 3 – Schéma SQL de la base Alpha